

SpringBoot Most Tricky Interview Questions in 2026

1. What problems does Spring Boot solve compared to Spring Framework?

Ans.  **Problem: Too Much Configuration in Spring Framework**

 **In Traditional Spring**

- You had to write **lots of XML or Java config**
- Example:
 - applicationContext.xml
 - dispatcher-servlet.xml
 - bean definitions
- Even a simple web app needed **hundreds of lines of configuration**

```
<bean id="dataSource"  
class="org.apache.commons.dbcp.BasicDataSourc  
e">
```

 **Time-consuming & error-prone**

 **How Spring Boot Solves It**

- **Auto-configuration**
- Based on:

- Classpath dependencies
- Existing beans
- Properties

@SpringBootApplication

```
public class MyApp {}
```

👉 **No XML, minimal Java config**

📌 **Interview line:**

Spring Boot reduces boilerplate configuration using auto-configuration.

2 Problem: Dependency Version Conflicts

✗ In Spring Framework

- You had to manually manage:
 - Spring version
 - Hibernate version
 - Jackson version
- One wrong version → **ClassNotFoundException**

✅ Spring Boot Solution: Starter Dependencies

- spring-boot-starter-web
 - spring-boot-starter-data-jpa
 - spring-boot-starter-security
- ```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

👉 SpringBoot internally manages **compatible versions**

📌 **Interview line:**

Spring Boot starters simplify dependency management and avoid version conflicts.

---

### 3 Problem: Complex Project Setup

#### ✗ In Spring Framework

To start a web app, you needed:

- Web.xml
- DispatcherServlet
- Component scanning
- View resolver
- Server setup (Tomcat manually)

😞 Too much setup before writing business logic.

#### ✅ Spring Boot Approach

- Embedded servers:
  - Tomcat
  - Jetty
  - Undertow

`java -jar myapp.jar`

👉 **No external server required**

📌 **Interview line:**

Spring Boot provides embedded servers, making applications standalone.

---

#### **Problem: Environment-Specific Configuration**

##### **In Spring**

- Separate XML files per environment
- Manual profile switching
- Hard to manage

#### **Spring Boot Solution: Profiles + Properties**

application-dev.properties

application-prod.properties

spring.profiles.active=dev

 Clean, centralized configuration

#### **Interview line:**

Spring Boot simplifies environment management using profiles and property files.

---

#### **Problem: No Production-Ready Features by Default**

##### **In Spring**

- No built-in:
  - Health checks
  - Metrics
  - Monitoring

- You had to integrate tools manually

### **Spring Boot Actuator**

spring-boot-starter-actuator

Provides:

- /actuator/health
- /actuator/metrics
- /actuator/info

### **Interview line:**

Spring Boot Actuator provides production-ready features out of the box.

---

## **Problem: Slower Development Speed**

### **In Spring**

- Restart server after every change
- Slower feedback loop

### **Spring Boot DevTools**

- Auto restart
- Live reload

### **Interview line:**

Spring Boot improves developer productivity with DevTools.

---

## **Problem: Hard to Follow Best Practices**

## In Spring

- Every team follows different structure
- No opinionated defaults

## Spring Boot Is Opinionated

- Standard project structure
- Sensible defaults
- Convention over configuration

## Interview line:

Spring Boot follows opinionated defaults to enforce best practices.

---

## One-Line Comparison (Very Important for Interviews)

### Spring Framework

Heavy configuration

Manual dependency management

External server needed

Slow setup

No production tools

### Spring Boot

Auto configuration

Starter dependencies

Embedded server

Rapid development

Actuator included

---

## **Short Interview Answer (Use This!)**

Spring Framework is powerful but requires a lot of configuration, dependency management, and server setup. Spring Boot solves these problems by providing auto-configuration, starter dependencies, embedded servers, and production-ready features. It follows convention over configuration, which makes development faster, easier, and more consistent.

### ***2. How does Spring Boot auto-configuration work internally?***

Ans. Spring Boot auto-configuration works by **automatically configuring beans based on classpath dependencies, existing beans, and application properties**, so developers don't have to configure everything manually.

---

#### **1 Entry Point: @SpringBootApplication**

@SpringBootApplication

```
public class MyApplication {}
```

Internally, this is a **combination of three annotations**:

@SpringBootConfiguration

@EnableAutoConfiguration

@ComponentScan

 **Key player for auto-configuration:**

 @EnableAutoConfiguration

---

## **2 @EnableAutoConfiguration – What Does It Do?**

@Import(AutoConfigurationImportSelector.class)

This annotation **imports**

**AutoConfigurationImportSelector**, which is responsible for:

- Finding all auto-configuration classes
- Loading them into the Spring context

 **Interview line:**

@EnableAutoConfiguration triggers the loading of auto-configuration classes using AutoConfigurationImportSelector.

---

## **3 Where Does Spring Boot Find Auto-Configuration Classes?**

Spring Boot looks into:



META-INF/spring.factories

Inside Spring Boot JARs:

org.springframework.boot.autoconfigure.EnableAutoConfiguration=\

org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration,\

org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration,\

org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration

👉 This file lists **all auto-configuration classes**

📌 **Interview line:**

Spring Boot uses spring.factories to locate auto-configuration classes.

---

🔖 **Auto-Configuration Classes Are Just @Configuration Classes**

Example:

@Configuration

@ConditionalOnClass(DataSource.class)

@ConditionalOnMissingBean(DataSource.class)

```
@EnableConfigurationProperties(DataSourceProperties
.class)
```

```
public class DataSourceAutoConfiguration {
}
```

So:

- They define **beans**
  - But only when **conditions are satisfied**
- 

## **5 The Real Magic: @Conditional Annotations**

Spring Boot auto-configuration heavily uses **conditional annotations**.

### **◆ Common Conditional Annotations**

<b>Annotation</b>	<b>Meaning</b>
@ConditionalOnClass	Class exists in classpath
@ConditionalOnMissingBean	Bean not already defined
@ConditionalOnProperty	Property is enabled
@ConditionalOnWebApplication	Web app detected

**Example:**

@ConditionalOnClass(Hibernate.class)

@ConditionalOnProperty(name =  
"spring.jpa.hibernate.ddl-auto")

👉 If conditions fail → configuration is **skipped**

📌 **Interview line:**

Auto-configuration is condition-based and activates only when required dependencies are present.

---

## 6 Property Binding: application.properties / yml

Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/test
spring.datasource.username=root
```

Spring Boot binds properties using:

@ConfigurationProperties(prefix = "spring.datasource")

👉 Automatically maps values to Java objects.

📌 **Interview line:**

Spring Boot uses @ConfigurationProperties to bind external configuration to beans.

---

## 7 Auto-Configuration Happens After User Configuration

Spring Boot follows this rule:

### **User configuration always wins**

- If you define your own bean:

@Bean

```
public DataSource dataSource() { }
```

- Auto-configuration backs off because of:

@ConditionalOnMissingBean

### **Interview line:**

Spring Boot auto-configuration is non-intrusive and backs off when user-defined beans exist.

---

## **Auto-Configuration Order**

- Auto-configuration runs **after**:
  - @ComponentScan
  - User @Configuration classes

Controlled by:

@AutoConfigureAfter

@AutoConfigureBefore

---

## **Debugging Auto-Configuration (VERY IMPORTANT)**

Enable debug mode:

debug=true

Or:

--debug

It shows:

- Which auto-configs were applied
- Which were skipped
- Why they were skipped



### **Interview line:**

Spring Boot provides a condition evaluation report to debug auto-configuration.

---



### **Internal Flow Summary (Explain Like This in Interviews)**

1. Application starts
2. @EnableAutoConfiguration is triggered
3. spring.factories is scanned
4. Auto-configuration classes are loaded
5. Conditional checks are evaluated
6. Beans are created if conditions match

## 7. User beans override auto-configured beans

---

### Interview Answer

Spring Boot auto-configuration works by scanning `spring.factories` for auto-configuration classes when `@EnableAutoConfiguration` is enabled. These classes are regular `@Configuration` classes annotated with conditional annotations like `@ConditionalOnClass` and `@ConditionalOnMissingBean`. Based on classpath dependencies, application properties, and existing beans, Spring Boot automatically configures beans while backing off if the user provides custom configurations.

### ***Q3. How does Spring Boot decide which auto-configuration to apply?***

Ans. Spring Boot decides which auto-configuration to apply by evaluating **conditional annotations** based on the **classpath**, **existing beans**, **application properties**, and **application type**.

---

### Step 1: Load All Auto-Configuration Candidates

- Spring Boot **does NOT blindly apply everything**
- First, it loads **all auto-configuration class names** from:

META-INF/spring.factories

Example entries:

- DataSourceAutoConfiguration
- WebMvcAutoConfiguration
- JpaRepositoriesAutoConfiguration

👉 These are only **candidates**, not yet applied.

📌 Interview line:

Spring Boot first loads auto-configuration candidates from spring.factories.

---

## **2 Step 2: Evaluate @Conditional Annotations (Decision Point)**

Each auto-configuration class is guarded by **conditions**.

🔥 **This is where the decision actually happens**

Spring Boot checks:

### **✓ A. Classpath Conditions (@ConditionalOnClass)**

@ConditionalOnClass(DataSource.class)

- If the class exists → possible
- If not → skipped

 Example:

- spring-boot-starter-jdbc present → DataSource auto-config applies
- Not present → ignored

 Interview line:

Auto-configuration depends on available classes in the classpath.

---

## B. Missing or Existing Beans (@ConditionalOnMissingBean)

@ConditionalOnMissingBean(DataSource.class)

- If **user already defined a bean** → auto-config backs off
- If not → auto-config creates it

 Interview line:

User-defined beans always override auto-configuration.

---

## C. Application Properties (@ConditionalOnProperty)



```
@ConditionalOnProperty(
 name = "spring.jpa.enabled",
 havingValue = "true",
 matchIfMissing = true
)
```

- Reads from:
  - application.properties
  - application.yml
  - environment variables

 Interview line:

Spring Boot enables or disables auto-configuration using properties.

---

## D. Application Type

### **(@ConditionalOnWebApplication)**

```
@ConditionalOnWebApplication(type = SERVLET)
```

- Servlet-based app → MVC auto-config
- Reactive app → WebFlux auto-config
- Non-web app → skipped

 Interview line:

Spring Boot applies different auto-configurations based on application type.

---

### **E. Profiles (@Profile)**

@Profile("dev")

- Auto-config only applies when that profile is active
- 

### **Step 3: Order & Dependencies Between Auto-Configurations**

Spring Boot controls order using:

@AutoConfigureBefore

@AutoConfigureAfter

@AutoConfigureOrder

 Ensures:

- DataSource loads before JPA
- JPA loads before Hibernate

 Interview line:

Auto-configuration order is managed to resolve dependencies correctly.

---

#### **Step 4: Property Binding Influences Final Behavior**

Even if auto-config is applied, **properties customize it**.

Example:

```
spring.datasource.url=jdbc:mysql://localhost/test
```

```
spring.jpa.hibernate.ddl-auto=update
```

Bound using:

```
@ConfigurationProperties
```



Interview line:

Properties fine-tune auto-configuration behavior after selection.

---

#### **Step 5: Condition Evaluation Report (Debug Mode)**

Run app with:

```
debug=true
```

You get:

- ✓ Applied auto-configs
- ✗ Skipped auto-configs
-  Reason for skipping



Interview line (very strong):

Spring Boot provides a condition evaluation report to explain auto-configuration decisions.

---

### **Internal Decision Flow (Explain This Clearly)**

1. Load auto-config candidates from spring.factories
  2. Evaluate conditional annotations
  3. Check classpath dependencies
  4. Check existing beans
  5. Check application properties
  6. Apply only matching auto-configurations
- 

### **Short Answer**

Spring Boot decides which auto-configuration to apply by evaluating conditional annotations on auto-configuration classes. It checks the classpath for required dependencies, verifies whether beans already exist, reads application properties, and detects the application type. Only when all conditions are satisfied does Spring Boot apply the corresponding auto-configuration, otherwise it skips it.

---

**Example(real world )**

## Real-world analogy

“Spring Boot behaves like a smart switchboard. It sees what dependencies you have, checks what you already configured, reads your properties, and only then turns on the required configurations.”

### ***3. Difference between application.properties vs application.yml?***

Ans. There is no functional difference between application.properties and application.yml in Spring Boot. Both are used for externalized configuration. The difference is mainly in syntax, readability, and structure.

---

#### **1** Format & Syntax Difference

application.properties (Key–Value Pair)

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/test

spring.datasource.username=root

- Flat structure
- Dot-separated keys
- Easy for small configurations

---

application.yml (YAML – Hierarchical)

server:

port: 8080

spring:

datasource:

url: jdbc:mysql://localhost:3306/test

username: root

- Indentation-based
- Hierarchical and clean
- Better for complex configs

 Interview line:

YAML provides a hierarchical structure, while properties files are flat.

#### ***4. What is @SpringBootApplication?***

Ans. @SpringBootApplication is a convenience annotation in Spring Boot that marks the main class of a Spring Boot application and enables auto-

configuration, component scanning, and Spring Boot configuration with a single annotation.

---

## 1 What Does @SpringBootApplication Contain Internally?

@SpringBootApplication is a meta-annotation made up of three core annotations:

@SpringBootApplication

Expands to:

@SpringBootConfiguration

@EnableAutoConfiguration

@ComponentScan

 Interview line:

@SpringBootApplication is a combination of three annotations.

---

## 2 @SpringBootConfiguration

- Specialized version of @Configuration
- Marks the class as a source of bean definitions
- Allows @Bean methods

@SpringBootConfiguration

```
public class MyAppConfig {}
```

 Interview line:

@SpringBootConfiguration indicates a Spring Boot configuration class.

---

### @EnableAutoConfiguration (Most Important)

- Enables Spring Boot auto-configuration
- Automatically configures beans based on:
  - Classpath dependencies
  - Existing beans
  - Application properties

Internally imports:

AutoConfigurationImportSelector

 Interview line:

@EnableAutoConfiguration enables Spring Boot's auto-configuration mechanism.

---

### @ComponentScan

- Scans components (@Component, @Service, @Repository, @Controller)



- Default scan base package = package of the main class

 Interview line:

@ComponentScan automatically detects Spring components.

 Common mistake:

If your main class is in the wrong package, beans won't be detected.

### ***Q5. What happens internally when you run a Spring Boot application ?***

Ans. When a Spring Boot application starts, it bootstraps the Spring environment, creates the application context, performs auto-configuration, initializes beans, starts the embedded server, and finally makes the application ready to serve requests.

---

#### **1** JVM Starts & main() Method Executes

```
public static void main(String[] args) {
 SpringApplication.run(MyApplication.class, args);
}
```

}

- JVM starts
- Control enters the main() method
- SpringApplication.run() is the real bootstrapper



Interview line:

Spring Boot starts from the SpringApplication.run() method.

---

## **2** SpringApplication Is Created

Internally, Spring Boot:

- Determines application type
  - Servlet
  - Reactive
  - Non-web
- Loads ApplicationContext type
  - AnnotationConfigServletWebServerApplicationContext
- Registers:
  - Initializers
  - Listeners



Interview line:

Spring Boot detects the application type and prepares the appropriate application context.

---

3

### Prepare Environment (Very Important)

Spring Boot creates a ConfigurableEnvironment and:

- Loads configuration from:
  - application.properties / yml
  - Environment variables
  - JVM arguments
  - Command-line arguments
- Resolves property priority



Interview line:

Spring Boot prepares the environment before creating the application context.

---

4

### ApplicationContext Is Created

- Based on application type:
  - Web → WebApplicationContext
- Context is created but not yet refreshed

 Interview line:

Spring Boot creates the Spring application context.

---

## **5** Auto-Configuration Is Triggered

Triggered by:

@EnableAutoConfiguration

Flow:

1. Reads auto-config classes from spring.factories
2. Evaluates @Conditional annotations
3. Applies matching configurations

 Interview line:

Auto-configuration runs based on classpath, properties, and existing beans.

---

## **6** Component Scanning & Bean Registration

- @ComponentScan scans base packages
- Registers:
  - @Component
  - @Service
  - @Repository

- @Controller
- Bean definitions are prepared

 Interview line:

Spring scans and registers all application components.

---

## **7** Context Refresh (Core Spring Lifecycle)

This is where real magic happens:

During context.refresh():

- BeanFactory is initialized
- Beans are instantiated
- Dependencies injected
- Aware interfaces executed
- BeanPostProcessors applied
- @PostConstruct executed

 Interview line:

Spring initializes all beans during context refresh.

---

## **8** Embedded Server Starts (Web Apps)

For web applications:

- Embedded Tomcat/Jetty/Undertow starts

- DispatcherServlet is registered
- Application listens on configured port

 Interview line:

Spring Boot starts an embedded web server automatically.

---

### Application Ready Events Fired

Spring Boot publishes lifecycle events:

- ApplicationStartedEvent
- ApplicationReadyEvent

Used for:

- Startup logic
- Warm-up tasks

```
@EventListener(ApplicationReadyEvent.class)
```

```
public void onReady() {}
```

 Interview line:

Spring Boot publishes lifecycle events during startup.

---

### Full Startup Flow (Perfect to Explain)

1. JVM starts

2. `main()` executes
  3. `SpringApplication.run()`
  4. Environment prepared
  5. `ApplicationContext` created
  6. Auto-configuration applied
  7. Beans created and injected
  8. Embedded server starts
  9. Application ready to handle requests
- 

### Short Answer

When a Spring Boot application runs, the JVM starts and executes the main method, which calls `SpringApplication.run()`. Spring Boot prepares the environment, loads configuration properties, determines the application type, and creates the appropriate application context. It then performs component scanning and auto-configuration, initializes and injects beans, starts the embedded server, and finally publishes application ready events indicating that the application is ready to serve requests.

---

Best visualization:

JVM



main()



SpringApplication.run()



Environment



ApplicationContext



Auto-Configuration



Bean Initialization



Embedded Server



READY

Closing Line for Video:

“Spring Boot doesn’t just start an app — it orchestrates an entire startup lifecycle.”



## ***Q6. What is CommandLineRunner vs ApplicationRunner?***

Ans. Both CommandLineRunner and ApplicationRunner are used to execute code **after the Spring Boot application has started**, but they differ in how they receive command-line arguments.

---

### **1 What Is CommandLineRunner?**

- Functional interface provided by Spring Boot
- Runs **after application context is loaded**
- Receives arguments as a **String array**

#### **Example:**

@Component

```
public class MyCommandLineRunner implements
CommandLineRunner {
```

@Override

```
public void run(String... args) {
```

```
 System.out.println("CommandLineRunner args: " +
Arrays.toString(args));
 }
}
```



Interview line:

CommandLineRunner receives command-line arguments as a raw String array.

---

## **2 What Is ApplicationRunner?**

- Also a functional interface
- Runs **after application startup**
- Receives arguments as an **ApplicationArguments object**

**Example:**

```
@Component
```

```
public class MyApplicationRunner implements
ApplicationRunner {
```

```
 @Override
```

```
 public void run(ApplicationArguments args) {
 System.out.println(args.getOptionNames());
 }
}
```

```
}
}
```



Interview line:

ApplicationRunner provides a structured way to access command-line arguments.

---



### Key Difference: Argument Handling

#### Running the app:

```
java -jar app.jar --name=Spring --debug=true test1
test2
```

#### CommandLineRunner

- Receives:

```
["--name=Spring", "--debug=true", "test1", "test2"]
```

#### ApplicationRunner

- Can separate:
  - Option args: --name, --debug
  - Non-option args: test1, test2

```
args.getOptionValues("name");
```

```
args.getNonOptionArgs();
```



Interview line:

ApplicationRunner offers better parsing of command-line arguments.

---

#### **Execution Order (Very Important)**

If you have multiple runners:

```
@Order(1)
```

```
@Component
```

```
class FirstRunner implements CommandLineRunner { }
```

```
@Order(2)
```

```
@Component
```

```
class SecondRunner implements ApplicationRunner { }
```

- Both support:
  - @Order
  - Ordered interface

 Interview line:

Execution order can be controlled using @Order.

---

#### **When Exactly Do They Run?**

- After:

- ApplicationContext is created
- Beans are initialized
- Before:
  - Application is fully “ready” for external traffic



Interview line:

Runners execute after context initialization but before the application is marked ready.

---

## Common Use Cases

Use Case	Preferred Runner
Simple startup logic	CommandLineRunner
Reading CLI options	ApplicationRunner
DB initialization	Either
Data loading	Either
Conditional startup tasks	ApplicationRunner

---

## Can We Use Both Together?



Yes

- Spring Boot executes **both**

- Order is based on @Order

 Interview line:

Both runners can coexist and execute in a defined order.

---

### Comparison Table (Perfect for Interviews)

Feature	CommandLineRunner	ApplicationRunner
Arguments type	String... args	ApplicationArguments
Argument parsing	Manual	Built-in
Option vs non-option args	No	Yes
Execution timing	After context load	After context load
Ordering support	Yes	Yes

---

## Short Answer

Both `CommandLineRunner` and `ApplicationRunner` are used to execute code after a Spring Boot application starts. The main difference is in argument handling. `CommandLineRunner` receives raw command-line arguments as a `String` array, while `ApplicationRunner` provides a structured `ApplicationArguments` object that separates option and non-option arguments. If argument parsing is needed, `ApplicationRunner` is preferred.

---

### Easy analogy:

“`CommandLineRunner` gives you raw input, `ApplicationRunner` gives you parsed input.”

### Closing line:

“Same purpose, different power level.”

## ***Q7. What is fat jar / uber jar?***

Ans. **What is a JAR?**

A **JAR (Java ARchive)** is a ZIP file that contains:

- Compiled .class files
- Resources (properties, XML, images)
- META-INF/MANIFEST.MF

Normally, a JAR **does not include its dependencies**.

---

## What is a FAT JAR / UBER JAR?

A **Fat JAR (also called Uber JAR)** is a **single executable JAR file** that contains:

- ✓ Your application classes
  - ✓ **All dependent libraries (JARs)**
  - ✓ Resources
  - ✓ A manifest with a Main-Class
- 👉 Result: **One JAR to run everything**

```
java -jar app-fat.jar
```

---

## Why is it called FAT / UBER?

- **Fat** → Large in size (contains all dependencies)
- **Uber** → “Above all” / “all-in-one”

Both mean the same thing in practice.

---

## Normal JAR vs FAT JAR



Feature	Normal JAR	Fat / Uber JAR
Contains dependencies	✗ No	✓ Yes
Size	Small	Large
Easy deployment	✗ Needs classpath	✓ One file
Executable directly	✗ Usually no	✓ Yes
Used in microservices	✗ Rare	✓ Very common

---

## Why do we need a FAT JAR?

### Problem with Normal JAR

```
java -cp app.jar:lib/* com.app.Main
```

- ✗ Dependency mismatch
- ✗ Classpath issues
- ✗ Environment problems

### Solution: FAT JAR

```
java -jar app-fat.jar
```

- ✓ No missing dependency
- ✓ Easy deployment
- ✓ Same behavior everywhere

## **FAT JAR vs WAR (Interview Favorite 🔥 )**

### **FAT JAR**

Self-contained

Runs with `java -jar`

Microservices

Embedded server

### **WAR**

Needs app server

Deployed to Tomcat

Monolith apps

External server

---

## **Interview One-Line Answer**

**A Fat JAR (Uber JAR) is a self-contained executable JAR that bundles the application along with all its dependencies, allowing it to run independently without requiring an external classpath.**

***Q8. How do you change log levels/server port at runtime?***

Ans.

Spring Boot supports this out of the box

✓ Using Actuator

Enable actuator dependency:

```
<dependency>
```

```
 <groupId>org.springframework.boot</groupId>
```

```
 <artifactId>spring-boot-starter-actuator</artifactId>
```

```
</dependency>
```

Expose the endpoint:

```
management.endpoints.web.exposure.include=loggers
```

Change log level at runtime

```
curl -X POST
```

```
http://localhost:8080/actuator/loggers/com.myapp \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"configuredLevel":"DEBUG"}'
```

✓ Takes effect immediately

✓ No restart

✓ Works for package / class level

---

View current log level

curl

http://localhost:8080/actuator/loggers/com.myapp

---

Supported levels

TRACE, DEBUG, INFO, WARN, ERROR, OFF

---

Why this works

- Logging frameworks (Logback / Log4j2) are runtime configurable
  - Spring Boot delegates changes dynamically
- 

**2** Changing Server Port at Runtime ❌ (NOT POSSIBLE in standard Spring Boot)

Why?

- Server port is bound when the embedded server starts
- Tomcat / Jetty / Netty cannot rebind port without restart
- Active connections would break

⚠️ server.port is a startup property, not runtime configurable.

---

What happens if you try?

Changing:

`server.port=9090`

✓ Value changes in environment

✗ Server still listens on old port

---

### 3 Common Interview Trick Question

Can we change `server.port` at runtime using Actuator / `@RefreshScope`?

✗ No

Even with:

- Spring Cloud Config
- `/actuator/refresh`

The server will NOT rebind the port.

***Q9. How do you optimize Spring Boot startup time?***

Ans. **1 Reduce Auto-Configuration (BIGGEST WIN)**

Spring Boot loads **many auto-configs you don't need.**

## ❌ Default behavior

Spring Boot scans & loads everything on classpath.

## ✅ Disable unused auto-configs

```
@SpringBootApplication(
 exclude = {
 DataSourceAutoConfiguration.class,
 SecurityAutoConfiguration.class
 }
)
```

Or:

```
spring.autoconfigure.exclude=\norg.springframework.boot.autoconfigure.security.servl
et.SecurityAutoConfiguration
```

📌 **Impact:** Huge startup improvement

---

## 2 Use Lazy Initialization (Spring Boot 2.2+)

Delays bean creation until first use.

```
spring.main.lazy-initialization=true
```

✓ Faster startup

❌ Slight first-request delay

 Best for **REST APIs**

---

### **Limit Component Scanning**

Avoid scanning the entire classpath.

```
@ComponentScan(basePackages = "com.myapp.api")
```

 Avoid:

```
@ComponentScan("com")
```

 Reduces classpath scanning time

---

### **Optimize Logging (Surprisingly Important)**

Logging slows startup a lot.

 **Bad**

```
logging.level.root=DEBUG
```

 **Good**

```
logging.level.root=INFO
```

```
logging.level.org.springframework=WARN
```

 Logging I/O is expensive during startup

---

### **Disable Unused Starters**

Each starter adds auto-configs.

✗ Example:

spring-boot-starter-data-jpa

If you only need JDBC:

spring-boot-starter-jdbc

📌 Less class loading → faster boot

---

## 6 Optimize JPA / Hibernate Startup

Hibernate is **slow to initialize**.

spring.jpa.open-in-view=false

spring.jpa.hibernate.ddl-auto=none

spring.jpa.properties.hibernate.temp.use\_jdbc\_metadata\_defaults=false

✓ Avoid schema checks

✓ Avoid metadata lookups

---

## 7 Use Actuator Carefully

Actuator endpoints add overhead.

management.endpoints.enabled-by-default=false

management.endpoint.health.enabled=true



 Enable only what you need

---

## JVM Startup Optimizations

### Use Class Data Sharing (CDS)

```
java -Xshare:on -jar app.jar
```

### Tune memory

```
-Xms256m -Xmx256m
```

 Especially helpful in **containers**

---

## Use Spring Context Caching (Tests)

For test startup optimization:

```
@SpringBootTest
```

```
@DirtiesContext(classMode =
ClassMode.AFTER_CLASS)
```

 Speeds up test suite dramatically

---

## Spring AOT & Native Images (Modern Way )

### Spring Boot 3 + GraalVM

- Ahead-of-Time processing
- Native compilation

mvn -Pnative native:compile

✓ Startup in **milliseconds**

✓ Lower memory

✗ Longer build time

📌 Used in **serverless & cold-start sensitive apps**

---

### ⚡ Real-World Priority Order

1. Disable auto-configs
  2. Lazy initialization
  3. Reduce component scan
  4. Logging optimization
  5. JPA tuning
  6. JVM tuning
  7. AOT / Native (advanced)
- 

### 🎯 Interview-Ready Answer

**Spring Boot startup time can be optimized by disabling unnecessary auto-configurations, enabling lazy bean initialization, limiting component scanning, and tuning logging and JPA settings. For further**

optimization, JVM tuning, Spring AOT, and GraalVM native images can reduce startup time significantly.

---

***Q10. Bonus Question(Can SpringBoot App runs on multiple servers or how we can change the server)?***

Ans.  **How to Change the Server in Spring Boot**

Spring Boot uses **embedded servers** by default.

 **Default: Apache Tomcat**

 **Supported Embedded Servers**

- **Tomcat** (default)
- **Jetty**
- **Undertow**

---

 **Change from Tomcat → Jetty**

**Step 1: Exclude Tomcat**

`<dependency>`

`<groupId>org.springframework.boot</groupId>`

`<artifactId>spring-boot-starter-web</artifactId>`

```
<exclusions>
 <exclusion>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-
tomcat</artifactId>
 </exclusion>
</exclusions>
</dependency>
```

## Step 2: Add Jetty

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

---

## Change from Tomcat → Undertow

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-
undertow</artifactId>
</dependency>
```

 Spring Boot automatically detects **which server is on the classpath** and starts it.

---

### Interview Trick Question

**Q:** What if Tomcat, Jetty, and Undertow are all present?

**A:**

Spring Boot follows **classpath priority**:

Tomcat → Jetty → Undertow

First detected server is used.

---

### **2** Can a Spring Boot Application Run on Multiple Servers?

 Short answer: **✗ One JVM = One embedded server instance**

But interviewers want the **full picture** 

---

### **✗ Not Possible (Inside One JVM)**

A single Spring Boot application:

- Runs on **one port**
- Uses **one embedded server**
- Handles requests via **one Servlet container**

You **cannot** run Tomcat + Jetty together in the same Boot app.

---

✅ **Possible (Real-World Scenarios)**

◆ **1. Multiple Instances (Horizontal Scaling)**

Same Spring Boot app runs on:

- Multiple JVMs
- Multiple servers / containers
- Different ports

Example:

Server 1 → localhost:8080

Server 2 → localhost:8081

Server 3 → localhost:8082

➡ Traffic managed via:

- Load Balancer (Nginx / AWS ALB)
- Kubernetes Service
- API Gateway

✔ **This is how Spring Boot apps scale in production**

---

◆ **2. Same App, Multiple Ports (Advanced / Rare)**

You *can* configure **multiple connectors** programmatically (Tomcat only):

@Bean

```
public ServletWebServerFactory servletContainer() {
```

```
 TomcatServletWebServerFactory tomcat = new
 TomcatServletWebServerFactory();
```

```
 tomcat.addAdditionalTomcatConnectors(createConnector(8081));
```

```
 return tomcat;
```

```
}
```

⚠ Rarely used and **not recommended** for microservices.

---

### ◆ 3. Multiple Applications on Same Server

One physical server:

- Multiple Spring Boot apps
- Different ports
- Each app has its own embedded server

✓ Very common in VM-based deployments.

---

### **3 Interview Summary**

**“A Spring Boot application runs with a single embedded server per JVM.**

**To scale, we run multiple instances of the same application behind a load balancer rather than running multiple servers inside one application.”**